



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего
образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий
Кафедра математического обеспечения и стандартизации информационных
технологий

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 3
по дисциплине

«Тестирование и верификация программного обеспечения»

Выполнили студенты группы *ИКБО-74-23 Команда ЗавозWW*

Кузьмин И.В.

Тыцкий Б.С.

Принял

Ильичев Г.П.

Практическая

«22» октября 2025 г.

работа выполнена

«Зачтено»

«__» _____ 2025 г.

Москва 2025

Техническое задание проекта «Рекомендательная система фильмов»

1. Аннотация

Краткое описание: Программа «Рекомендательная система фильмов» разработана в учебных целях для демонстрации различных методологий тестирования

2. Введение

- Основание для выполнения работы: практическое задание по дисциплине «Программная инженерия».
- Цель работы: закрепить навыки разработки и тестирования программного обеспечения.
- Область применения: учебный процесс, демонстрация методологии тестирования и построения рекомендательных систем.

3. Техническое задание

3.1 Цель проекта

Целью практической работы является создание учебного приложения «**film_recommend**», реализующего простую рекомендательную систему фильмов. Программа используется для демонстрации методологий “TDD”, “ATDD”, “BDD” и “SDD”

3.2 Задачи

- Разработать программу по варианту(53)
- Протестировать программу на различные методологии

3.3 Реализованный функционал

Приложение «**film_recommend**» поддерживает следующие операции:

- добавление оценки фильму пользователем;
- получение рекомендаций на основе ранее оцененных фильмов;
 - фильтрация фильмов по жанрам;
 - вывод списка доступных фильмов;

3.4 Архитектура

- Основной программный продукт реализован в модуле `film_recommend.py`.
- Основной программный продукт написан на языке Python
- Дополнительно подготовлена документация, включающая описание функций, инструкции по запуску и тестированию.

3.5 Процесс тестирования

В ходе работы был реализован следующий процесс тестирования:

1. Протестировать функционал программы на полноценную эксплуатацию
2. Подробно расписать шаги по всем подходам к тестированию

4. Результаты

- Рабочее приложение с консольным интерфейсом.
- Набор тестов, демонстрирующих процесс поиска и исправления ошибок.
- Документация

5. Выводы

- был отработан полный цикл разработки и тестирования программного продукта с использованием методологий TDD, ATDD, BDD и SDD;
- продемонстрировано взаимодействие ролей «Разработчик» и «Тестировщик» в учебном процессе;
- получен практический опыт в модульном тестировании, включая написание тестов, фиксацию ошибок, их исправление и повторную проверку;
- закреплены навыки структурирования кода, документирования и подготовки отчета.

6. Первоначальная версия класса без функций

```
class Recommender:
    def __init__(self):
        self.catalog = {
            "Терминатор 2": "Научная фантастика",
            "Чужой": "Научная фантастика",
            "The Matrix": "Научная фантастика",
            "Бойцовский клуб": "Драма",
            "Форрест гамп": "Драма",
            "Криминальное чтиво": "Драма"
        }
        self.ratings = {}
        self.popular = ["Топ-10 популярных фильмов"]
```

7. Методологии разработки и тестирования

7.1 Test-Driven Development (TDD)

Методология разработки через тестирование предполагает написание модульных тестов до реализации функционала.

- На первом этапе формулируются тесты для ключевых функций: добавления оценки, получения рекомендаций, фильтрации по жанрам.
- Тесты изначально не проходят, так как код еще не реализован.
- Затем создается минимальная реализация функций, обеспечивающая успешное прохождение тестов.
- После этого проводится рефакторинг кода с сохранением корректности.

7.1.1 Test-Driven Development по шагам - тесты для расчета рекомендаций и обработки оценок

7.1.2 Создание теста

Первый шаг в TDD — написать тест, описывающий ожидаемое поведение функции. В нашем случае создаются тесты для обработки оценки фильма и расчета рекомендаций по жанру в приложении `film_recommend`.

```
import unittest
from film_Recommend import Recommender

class TestFilmRecommend(unittest.TestCase):
    def test_add_rating(self):
        rec = Recommender()
        rec.add_rating("user1", "Терминатор 2", 3)
        self.assertEqual(rec.ratings["user1"]["Терминатор 2"], 2)

    def test_recommendations(self):
        rec = Recommender()
        rec.add_rating("user1", "The Matrix", 1)
        recs = rec.get_recommendations("user1")
        self.assertIn("Терминатор 2", recs)
```

```

self.assertIn("Чужой", recs)

if __name__ == "__main__":
    unittest.main()

```

Сформированные тесты:

- test_add_rating: проверяет, что оценка фильма корректно сохраняется.
- test_recommendations: проверяет, что при высокой оценке Sci-Fi фильма рекомендуются другие фильмы того же жанра.

7.1.3 Запуск тестов

На этом этапе запускаются тесты. Если функциональность еще не реализована или реализована частично, тесты должны провалиться — это ожидаемое поведение, подтверждающее корректность постановки тестов.

Запуск:

```

PS C:\Users\kuzmi\Desktop\bb> python -m unittest test_film
EE
=====
ERROR: test_add_rating (test_film.TestFilmRecommend.test_add_rating)
-----
Traceback (most recent call last):
  File "C:\Users\kuzmi\Desktop\bb\test_film.py", line 7, in test_add_rating
    rec.add_rating("user1", "Терминатор 2", 5)
    ^^^^^^^^^^^^^^^^^
AttributeError: 'Recommender' object has no attribute 'add_rating'

=====
ERROR: test_recommendations (test_film.TestFilmRecommend.test_recommendations)
-----
Traceback (most recent call last):
  File "C:\Users\kuzmi\Desktop\bb\test_film.py", line 12, in test_recommendations
    rec.add_rating("user1", "The Matrix", 5)
    ^^^^^^^^^^^^^^^^^
AttributeError: 'Recommender' object has no attribute 'add_rating'

-----
Ran 2 tests in 0.114s

```

Рисунок 1 - провал тестов

Ожидаемый результат: падение тестов, который мотивирует к минимальной реализации.

7.1.4 Реализация функции

Реализуется минимальный объем кода в `film_recommend.py`, достаточный для успешного прохождения тестов. Ниже показаны ключевые методы.

```
def add_rating(self, user, movie, rating):
    if rating < 0 or rating > 3:
        raise ValueError("Неверное число")
    if movie not in self.catalog:
        raise ValueError("")
    self.ratings.setdefault(user, {})[movie] = rating
```

7.1.5 Повторный запуск тестов

Тесты запускаются повторно, чтобы убедиться, что реализованная функциональность проходит проверки.



```
PS C:\Users\kuzmi\Desktop\bb> python -m unittest test_film
..
-----
Ran 2 tests in 0.001s
OK
```

Рисунок 2 - тесты пройдены

7.1.6 Рефакторинг

После успешного прохождения тестов проводится рефакторинг кода для улучшения читаемости, поддержки и эффективности без изменения поведения (тесты должны продолжать проходить).

```
def add_rating(self, user, movie, rating):
    if rating < 0 or rating > 3:
        raise ValueError("Оценка должна быть от 0 до 5")
    if movie not in self.catalog:
        raise ValueError("Фильм отсутствует в каталоге")
    self.ratings.setdefault(user, {})[movie] = rating
```


7.2 Acceptance Test-Driven Development (ATDD)

Методология разработки через приемочные тесты ориентирована на проверку соответствия приложения бизнес-требованиям.

- Совместно с «заказчиком» формулируются сценарии использования.
- Эти сценарии превращаются в автоматизированные приемочные тесты.
- Реализация считается завершенной, если все приемочные тесты проходят успешно.

7.2.1 Acceptance Test-Driven Development по шагам - сценарии для проверки корректности рекомендаций.

7.2.2 Формулировка требований вместе с заказчиком и аналитиками

На встрече обсуждаются бизнес-требования к системе рекомендаций:

- Если пользователь оценил фильм жанра «Научная фантастика» на 4 или 5, система должна рекомендовать ему другие фильмы того же жанра.
- Если пользователь оценил фильм жанра «Драма» на 4 или 5, система должна рекомендовать ему другие драмы.
- Если пользователь еще не поставил ни одной оценки, система должна предложить список популярных фильмов.

7.2.3 Создание приемочных тестов

На основании требований формируются приемочные тесты

```
import unittest
from film_Recommend import Recommender

class TestAcceptanceFilmRecommend(unittest.TestCase):
    def setUp(self):
        self.rec = Recommender()

    def test_recommendations_scifi(self):
        """Пользователь оценил Sci-Fi фильм"""
        self.rec.add_rating("user1", "The Matrix", 5)
        recs = self.rec.get_recommendations("user1")
        self.assertIn("Терминатор 2", recs)
        self.assertIn("Чужой", recs)

    def test_recommendations_drama(self):
        """Пользователь оценил драму"""
        self.rec.add_rating("user1", "Форрест гамп", 4)
        recs = self.rec.get_recommendations("user1")
        self.assertIn("Бойцовский клуб", recs)
        self.assertIn("Криминальное чтиво", recs)

    def test_recommendations_no_ratings(self):
        """Пользователь не оценил ни одного фильма"""
        recs = self.rec.get_recommendations("user2")
        self.assertIn("Топ-10 популярных фильмов", recs)

if __name__ == "__main__":
    unittest.main()
```

7.2.4 Запуск теста

При первом запуске тестов, тест упадет

```
===== КЕСТАКТ: C:/Users/kuzmi/Desktop/bb/test_film_rec.py =====
EEE
=====
ERROR: test_recommendations_drama (__main__.TestAcceptanceFilmRecommend.test_recommendations_drama)
Пользователь оценил драму
-----
Traceback (most recent call last):
  File "C:/Users/kuzmi/Desktop/bb/test_film_rec.py", line 18, in test_recommendations_drama
    recs = self.rec.get_recommendations("user1")
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'Recommender' object has no attribute 'get_recommendations'
=====
ERROR: test_recommendations_no_ratings (__main__.TestAcceptanceFilmRecommend.test_recommendations_no_ratings)
Пользователь не оценил ни одного фильма
-----
Traceback (most recent call last):
  File "C:/Users/kuzmi/Desktop/bb/test_film_rec.py", line 24, in test_recommendations_no_ratings
    recs = self.rec.get_recommendations("user2")
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'Recommender' object has no attribute 'get_recommendations'
=====
ERROR: test_recommendations_scifi (__main__.TestAcceptanceFilmRecommend.test_recommendations_scifi)
Пользователь оценил Sci-Fi фильм
-----
Traceback (most recent call last):
  File "C:/Users/kuzmi/Desktop/bb/test_film_rec.py", line 11, in test_recommendations_scifi
    recs = self.rec.get_recommendations("user1")
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'Recommender' object has no attribute 'get_recommendations'
-----
Ran 3 tests in 0.022s
FAILED (errors=3)
```

Рисунок 3 - тест упал

7.2.5 Написание функции по сценариям

Пишем код функции `get_Recommend`, который проходит приемочные тесты

```
def get_recommendations(self, user):
    if user not in self.ratings or not self.ratings[user]:
        return self.popular

    preferred_genres = {
        self.catalog[movie]
        for movie, rating in self.ratings[user].items()
        if rating >= 4
    }
    recs = [
        movie for movie, genre in self.catalog.items()
        if genre in preferred_genres and movie not in
self.ratings[user]
    ]
    return recs if recs else self.popular
```

7.2.6 Повторный тест

```
PS C:\Users\kuzmi\Desktop\bb> python -m unittest test_film_rec
...
-----
Ran 3 tests in 0.000s

OK
```

Рисунок 4 - пройденный тест

7.3 Behavior-Driven Development (BDD)

Behavior-Driven Development (BDD), или «разработка через поведение», - это методология, которая описывает ожидаемое поведение системы в формате, понятном всем участникам: разработчикам, тестировщикам и бизнес-аналитикам.

BDD развивает идеи TDD и делает акцент на сценариях использования в формате Given-When-Then.

7.3.1 Behavior-Driven Development по шагам - сценарий «Given фильм оценен, When запрашиваю рекомендации, Then получаю похожие фильмы»

7.3.2 Написание сценариев

Написание сценариев на языке Gherkin в структуре Given-When-Then

Feature: Рекомендации фильмов

Чтобы получать интересные фильмы

Как пользователь системы

Я хочу видеть рекомендации на основе моих оценок

Scenario: Рекомендации по жанру "Научная фантастика"

Given пользователь оценил фильм "The Matrix" на 5

When он запрашивает рекомендации

Then система предлагает фильмы "Терминатор 2" и "Чужой"

Scenario: Рекомендации по жанру "Драма"

Given пользователь оценил фильм "Форрест гамп" на 4

When он запрашивает рекомендации

Then система предлагает фильмы "Бойцовский клуб" и "Криминальное чтиво"

Scenario: Нет оцененных фильмов

Given пользователь еще не оценил ни одного фильма

When он запрашивает рекомендации

Then система предлагает "Топ-10 популярных фильмов"

7.3.3 Автоматизация сценария с использованием step definitions

В файле `recommend_steps.py` прописаны шаги теста, эмулирующие:

- добавление оценки фильму,
- запрос рекомендаций пользователем,
- проверку того, что система возвращает ожидаемые фильмы.

```
from behave import given, when, then
from film_Recommend import Recommender

@given('пользователь оценил фильм "{movie}" на {rating:d}')
def step_impl(context, movie, rating):
    if not hasattr(context, "rec"):
        context.rec = Recommender()
    context.rec.add_rating("user1", movie, rating)

@given('пользователь еще не оценил ни одного фильма')
def step_impl(context):
    context.rec = Recommender()

@when('он запрашивает рекомендации')
def step_impl(context):
    context.result = context.rec.get_recommendations("user1")

@then('система предлагает фильмы "{movie1}" и "{movie2}"')
def step_impl(context, movie1, movie2):
    assert movie1 in context.result, f"{movie1} не найден в {context.result}"
    assert movie2 in context.result, f"{movie2} не найден в {context.result}"

@then('система предлагает "{movie}"')
def step_impl(context, movie):
    assert movie in context.result, f'{movie} не найден в {context.result}'
```

На данном этапе сценарии запускаются через `behave`. Так как функциональность рекомендаций еще не реализована полностью (или содержит преднамеренные ошибки), тесты проваливаются. Это подтверждает, что сценарии действительно проверяют бизнес-требования и служат основой для дальнейшей разработки.

7.3.4 Создание функции

```
def add_rating(self, user, movie, rating):
    if rating < 0 or rating > 5:
        raise ValueError("Оценка должна быть от 0 до 5")
    if movie not in self.catalog:
        raise ValueError("Фильм отсутствует в каталоге")
    self.ratings.setdefault(user, {})[movie] = rating

def get_recommendations(self, user):
    if user not in self.ratings or not self.ratings[user]:
        return self.popular

    preferred_genres = {
        self.catalog[movie]
        for movie, rating in self.ratings[user].items()
        if rating >= 4
    }

    recs = [
        movie for movie, genre in self.catalog.items()
        if genre in preferred_genres and movie not in self.ratings[user]
    ]
    return recs if recs else self.popular

def filter_by_genre(self, genre):
    return [movie for movie, g in self.catalog.items() if g.lower() ==
genre.lower()]
```

7.3.5 Повторный запуск тестов

```
PS C:\Users\kuzmi\Desktop\bb> behave features
USING RUNNER: behave.runner:Runner
Feature: Рекомендации фильмов # features/recommend.feature:1
  Чтобы получать интересные фильмы
  Как пользователь системы
  Я хочу видеть рекомендации на основе моих оценок
    Then система предлагает фильмы "Терминатор 2" и "Чужой" # features/steps/recommend_steps.py:18 0.000s
    Then система предлагает фильмы "Терминатор 2" и "Чужой" # features/steps/recommend_steps.py:18
    Then система предлагает фильмы "Бойцовский клуб" и "Криминальное чтиво" # features/steps/recommend_steps.py:18 0.000s
    Then система предлагает фильмы "Бойцовский клуб" и "Криминальное чтиво" # features/steps/recommend_steps.py:18
    Then система предлагает "Топ-10 популярных фильмов" # features/steps/recommend_steps.py:23 0.000s
    Then система предлагает "Топ-10 популярных фильмов" # features/steps/recommend_steps.py:23
1 feature passed, 0 failed, 0 skipped
3 scenarios passed, 0 failed, 0 skipped
9 steps passed, 0 failed, 0 skipped
Took 0min 0.004s
```

Рисунок 5 - пройденный тест

7.3.6 Рефакторинг

Не требуется так как функции достаточно проработаны

7.4 Specification by Example (SDD)

Методология разработки через спецификации на примерах предполагает фиксацию требований в виде таблиц с конкретными входными данными и ожидаемыми результатами.

7.4.1 Specification by Example по шагам - спецификации с таблицами оценок и результатов.

7.4.2 Документирование примеров

Таблица 1 исходные действия и ожидаемые результаты

Пользовательские действия	Ожидаемый результат (рекомендации)
Оценил фильм <i>The Matrix</i> на 5	«Терминатор 2», «Чужой»
Оценил фильм <i>Форрест Гамп</i> на 4	«Бойцовский клуб», «Криминальное чтиво»
Не оценил ни одного фильма	«Топ-10 популярных фильмов»
Отфильтровал по жанру «Драма»	«Форрест Гамп», «Бойцовский клуб», «Криминальное чтиво»
Отфильтровал по жанру «Научная фантастика»	«The Matrix», «Терминатор 2», «Чужой»

7.4.3 Создание спецификаций

Feature: Рекомендательная система фильмов

Чтобы получать интересные фильмы

Как пользователь системы

Я хочу видеть рекомендации на основе моих оценок и фильтровать по жанрам

Scenario Outline: Получение рекомендаций по жанру

Given пользователь оценил фильм "<rated_movie>" на <rating>

When он запрашивает рекомендации

Then система предлагает фильмы "<expected1>" и "<expected2>"

Scenario: Нет оцененных фильмов

Given пользователь еще не оценил ни одного фильма

When он запрашивает рекомендации

Then система предлагает "Топ-10 популярных фильмов"

Scenario Outline: Фильтрация фильмов по жанру

Given в каталоге есть фильмы разных жанров

When пользователь фильтрует по жанру "<genre>"

Then система возвращает список "<movies>"

7.4.4 Автоматизированные тесты

```
from behave import given, when, then
from film_Recommend import Recommender

@given('пользователь оценил фильм "{movie}" на {rating:d}')
def step_impl(context, movie, rating):
    context.rec = Recommender()
    context.rec.add_rating("user1", movie, rating)

@given('пользователь еще не оценил ни одного фильма')
def step_impl(context):
    context.rec = Recommender()

@given('в каталоге есть фильмы разных жанров')
def step_impl(context):
    context.rec = Recommender()

@when('он запрашивает рекомендации')
def step_impl(context):
    context.result = context.rec.get_recommendations("user1")

@when('пользователь фильтрует по жанру "{genre}"')
def step_impl(context, genre):
    context.result = context.rec.filter_by_genre(genre)

@then('система предлагает фильмы "{movie1}" и "{movie2}"')
def step_impl(context, movie1, movie2):
    assert movie1 in context.result
    assert movie2 in context.result

@then('система предлагает "{movie}"')
def step_impl(context, movie):
    assert movie in context.result

@then('система возвращает список "{movies}"')
def step_impl(context, movies):
    expected = [m.strip() for m in movies.split(",")]
    for m in expected:
        assert m in context.result
```

7.4.5 Написание логики

```
class Recommender:
    def __init__(self):
        self.catalog = {
            "Терминатор 2": "Научная фантастика",
            "Чужой": "Научная фантастика",
            "The Matrix": "Научная фантастика",
            "Бойцовский клуб": "Драма",
            "Форрест гамп": "Драма",
            "Криминальное чтиво": "Драма"
        }
        self.ratings = {}
        self.popular = ["Топ-10 популярных фильмов"]

    def add_rating(self, user, movie, rating):
        if rating < 0 or rating > 5:
            raise ValueError("Оценка должна быть от 0 до 5")
        if movie not in self.catalog:
            raise ValueError("Фильм отсутствует в каталоге")
        self.ratings.setdefault(user, {})[movie] = rating

    def get_recommendations(self, user):
        if user not in self.ratings or not self.ratings[user]:
            return self.popular

        preferred_genres = {
            self.catalog[movie]
            for movie, rating in self.ratings[user].items()
            if rating >= 4
        }

        recs = [
            movie for movie, genre in self.catalog.items()
            if genre in preferred_genres and movie not in
self.ratings[user]
        ]
        return recs if recs else self.popular

    def filter_by_genre(self, genre):
        return [movie for movie, g in self.catalog.items() if g.lower()
== genre.lower()]
```

7.5 Финальное приложение полученное в ходе тестирования

```
class Recommender:
    def __init__(self):
        self.catalog = {
            "Терминатор 2": "Научная фантастика",
            "Чужой": "Научная фантастика",
            "The Matrix": "Научная фантастика",
            "Бойцовский клуб": "Драма",
            "Форрест гамп": "Драма",
            "Криминальное чтиво": "Драма"
        }
        self.ratings = {}
        self.popular = ["Топ-10 популярных фильмов"]

    def add_rating(self, user, movie, rating):
        if rating < 0 or rating > 5:
            raise ValueError("Оценка должна быть от 0 до 5")
        if movie not in self.catalog:
            raise ValueError("Фильм отсутствует в каталоге")
        self.ratings.setdefault(user, {})[movie] = rating

    def get_recommendations(self, user):
        if user not in self.ratings or not self.ratings[user]:
            return self.popular

        preferred_genres = {
            self.catalog[movie]
            for movie, rating in self.ratings[user].items()
            if rating >= 4
        }

        recs = [
            movie for movie, genre in self.catalog.items()
            if genre in preferred_genres and movie not in self.ratings[user]
        ]
        return recs if recs else self.popular

    def filter_by_genre(self, genre):
        return [movie for movie, g in self.catalog.items() if g.lower() ==
genre.lower()]
```

```
def main():
    rec = Recommender()
    user = input("Введите имя пользователя: ")

    while True:
        print("\nМеню:")
        print("1. Поставить оценку фильму")
        print("2. Получить рекомендации")
        print("3. Фильтровать фильмы по жанру")

        choice = input("Выберите действие: ")

        if choice == "1":
            print("Доступные фильмы:", ", ".join(rec.catalog.keys()))
            movie = input("Введите название фильма: ")
            try:
                rating = float(input("Введите оценку (0-5): "))
                rec.add_rating(user, movie, rating)
                print(f"Оценка {rating} для фильма '{movie}' сохранена.")
            except Exception as e:
                print("Ошибка:", e)

        elif choice == "2":
            recs = rec.get_recommendations(user)
            print("Рекомендации:", ", ".join(recs))

        elif choice == "3":
            genre = input("Введите жанр (например, Научная фантастика или Драма): ")
            movies = rec.filter_by_genre(genre)
            if movies:
                print("Фильмы жанра", genre, ":", ", ".join(movies))
            else:
                print("Фильмы данного жанра не найдены.")

        else:
            print("Некорректный выбор. Попробуйте снова.")

if __name__ == "__main__":
    main()
```

8. Результаты тестирования и анализа

8.1. Применение методологии TDD (Test-Driven Development)

- В рамках TDD процесс разработки строился по циклу *«красный — зеленый — рефакторинг»*.
- Сначала формулировались юнит-тесты, которые изначально завершались неуспешно.
- Затем реализовывалась минимальная логика, обеспечивающая прохождение тестов.
- После успешного выполнения тестов код подвергался рефакторингу при сохранении корректности поведения.
- Такой подход позволил выявлять ошибки на ранних этапах и гарантировать, что каждая новая функция системы сопровождалась проверкой.

8.2. Применение методологии ATDD (Acceptance Test-Driven Development)

- В ATDD ключевым элементом стало согласование критериев приемки с «заказчиком» до начала реализации.
- Для рекомендательной системы это означало формулирование требований в терминах бизнес-ценности: *«Если пользователь оценил фильм жанра „Научная фантастика“ на высокую оценку, система должна предложить другие фильмы того же жанра»*.
- Такие критерии были зафиксированы в виде приемочных тестов, которые служили ориентиром для разработчиков и тестировщиков.

8.3. Применение методологии BDD (Behavior-Driven Development)

- BDD позволила описывать поведение системы на естественном языке с использованием формата Gherkin.
- Сценарии вида *Given-When-Then* обеспечили прозрачность требований и сделали спецификации понятными как для разработчиков, так и для пользователей.

- Например: *Given* пользователь оценил фильм «The Matrix» на 5, *When* он запрашивает рекомендации, *Then* система предлагает фильмы «Терминатор 2» и «Чужой».
- Такой подход упрощает коммуникацию и обеспечил воспроизводимость тестов.

8.4. Применение методологии SDD (Specification by Example / Specification-Driven Development)

- В SDD спецификации фиксировались в виде таблиц примеров, связывающих входные данные и ожидаемые результаты.
- Для рекомендательной системы были составлены таблицы:
 - *Исходные данные*: оцененный фильм и выставленная оценка.
 - *Ожидаемый результат*: список фильмов, которые система должна предложить.
- Эти таблицы стали основой для автоматизированных тестов и позволили устранить двусмысленность требований.

8.5. Общий анализ результатов

- Все методологии продемонстрировали эффективность в интеграции тестирования в процесс разработки.
- TDD обеспечила надежность на уровне модулей.
- ATDD позволила согласовать ожидания с точки зрения бизнеса.
- BDD сделала спецификации доступными для всех участников проекта.
- SDD устраняет двусмысленность требований за счет конкретных примеров.

9. Заключение

9.1. Общая оценка качества программного продукта

- Программный продукт реализует все заявленные функции: выставление оценок, формирование рекомендаций, фильтрацию по жанрам.
- Система прошла тестирование по заранее определенным спецификациям и показала стабильность работы.

9.2. Оценка качества документации

- Документация представлена в виде таблиц примеров и спецификаций на естественном языке.
- Такой формат обеспечивает прозрачность требований и облегчает коммуникацию между разработчиками, тестировщиками и заказчиками.

9.3. Выводы о проделанной работе

- Использование методологий TDD, BDD и SDD позволило интегрировать тестирование в процесс разработки и повысить качество кода.
- Система готова к дальнейшему развитию: возможно внедрение более сложных алгоритмов рекомендаций (например, коллаборативной фильтрации) и расширение функционала.
- Полученный опыт подтверждает эффективность применения практик разработки через тестирование и спецификации.